AI 채용 플랫폼 — 그래프 구조

LangGraph 기반 질문 생성·면접 진행·평가 설계 정리 · 2026-11-10 리팩터링 반영 (ADR-0016, 0020, 0022, 0025, 0027)

1. 전체 흐름

LangGraph 그래프는 3개다. 면접 전·중에 질문을 만드는 ① 질문 생성 그래프, 지원자 1명의 면접을 진행하는 ② 면접 진행 그래프, 제출 뒤 리포트를 만드는 ③ 평가 그래프다. 회사 자료 검색(RAG)은 ①의 노드 안에서 도는 함수 루프다. 모든 그래프는 작업 대기열(DB **jobs** 테이블)의 작업이나 면접 API에서 실행된다.

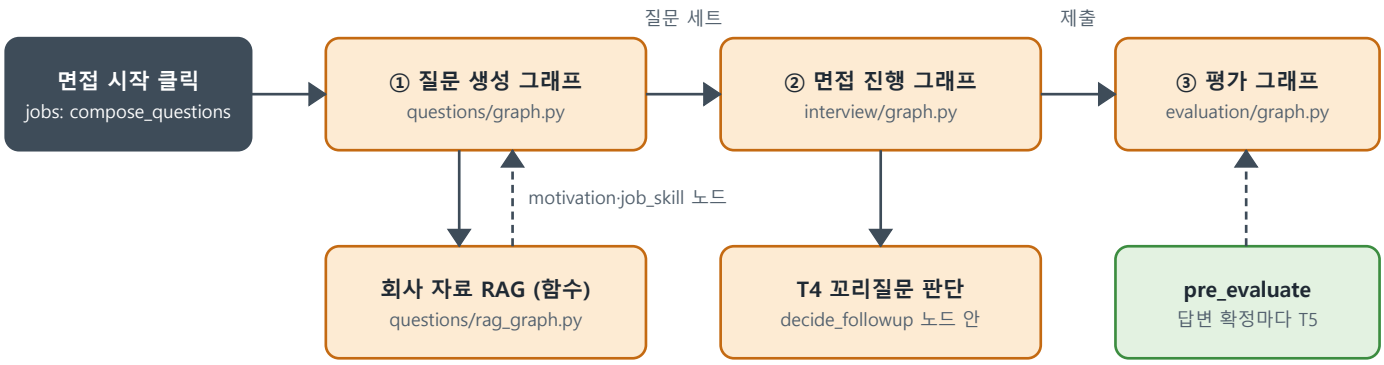


그림 1. 그래프와 작업의 관계. ①은 compose_questions 작업(live 모드면 면접 중 다음 질문도)으로, ③은 evaluate_application 작업으로 실행된다.

공통 원칙

- 다음 단계(엣지)는 코드가 정한다. 모델은 노드 안에서 질문·판단·점수만 만들고, 어느 노드로 갈지는 고르지 않는다.
- 모델을 부르는 동안 DB 트랜잭션을 열지 않는다. 노드는 짧게 읽고 닫은 뒤 모델을 부르고, 결과는 별도의 짧은 트랜잭션으로 쓴다(테스트가 모든 LLM·임베딩 호출 시점에 열린 트랜잭션 0개를 확인).
- 업무 DB가 진실이다. 노드는 DB에 이미 결과가 있으면 모델을 부르지 않는다. 그래서 같은 단계가 다시 실행돼도(크래시 후 재개) 모델 호출·저장이 한 번만 일어난다.
- 그래프 상태에는 id·코드·개수·점수만 둔다. 답변 원문과 인용 문장은 넣지 않는다(질문 생성 상태의 질문 문장은 실행이 끝나면 삭제).

재시도는 층마다 하나의 책임

층	무엇을	상한
run_task (노드 안, ai/tasks/base.py)	형식 재요청, 검증 실패 재생성	형식 2회(T2는 1회), 재생성은 작업별 1~3회
그래프	자체 재시도 루프 없음. 실패한 노드에서 멈춘다	—
jobs (DB 작업 큐)	작업 재시도. 같은 job이면 같은 스레드를 이어 실행해 끝난 노드는 다시 부르지 않는다	작업별 시도 수, 모델 서버 장애는 시도 미소모
Celery + Redis	jobs의 작업을 자원별 큐로 전달만 한다	재시도하지 않음

2. ① 질문 생성 그래프 — questions/graph.py

호출자는 **questions.graph.run()** 하나만 쓴다. 분기 없이 일직선이며, 본질문 10개(인성2·동기1·직무3·경험3·협업1, 커스텀 질문이 있으면 그 자리)를 칸별로 채운다.

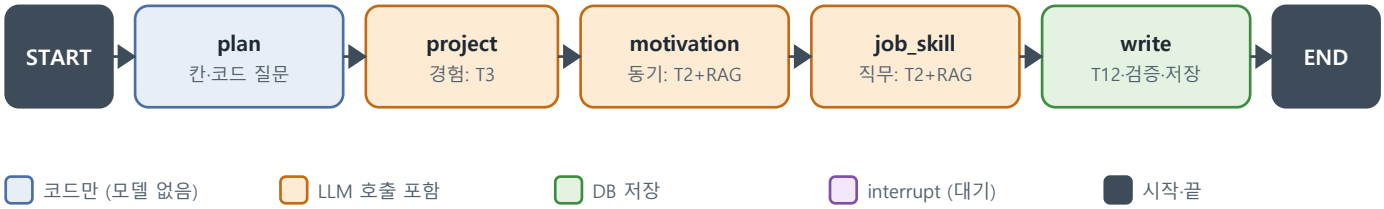


그림 2. 질문 생성 그래프. thread_id = compose:<지원 id>:<평가 설정 버전>. 상태: application_id, settings_id, live, picked, qset_id

노드	하는 일	모델	실패하면
plan	칸 배정, 커스텀 질문, 인성·협업 질문(이력서 경험 + 고정 문장). 모델 없음	없음	—
project (경험)	T3: 이력서에서 물을 부분을 원문 인용으로 고름 → 코드가 문장 조립	T3	코드가 만드는 경험 질문(안 물은 경력·프로젝트 이름 + 고정 문장)
motivation (동기)	회사 자료 RAG로 T2 생성	T2	공고 기반 고정 동기 질문
job_skill (직무)	공고·회사 자료 RAG로 T2 생성, 지원자 수준별 난이도	T2	공고 기반 대체 질문
write	은행 질문이 있을 때만 T12 각색 → 칸·중복 검증 → 짧은 트랜잭션: 이전 세트 대체, 불변 질문 행, tts_pregen 등록	T12(선택)	작업 재시도

검증과 제한적 재생성

- 검사: 유형(동기·직무 여부), 수준·난이도(TOO_HARD·MANY_TOPICS·TOO_LONG), 근거(인용 원문 일치·자료 복사·없는 용어), 중복·같은 근거, 명확성(qc-1: 한자·가나 등 외국 문자 FOREIGN_SCRIPT, 물음표·조건이 2개 이상인 겹질문 NESTED).
- 실패한 질문만 다시 만든다. 재생성 요청에 질문 번호별 실패 코드를 담고 통과한 질문은 고정한다. 재생성은 1회. 그래도 모자란 칸은 코드가 만든 대체 질문(source=TEMPLATE)으로 채운다.
- 버린 질문은 코드별 개수로만 로그에 남긴다(question_reject). 질문 원문은 남기지 않는다.

live 모드 (기본, ADR-0022)

- 면접 전에는 계획과 첫 질문만 만든다. 본질문 N이 나오면 N+1을 백그라운드에서 미리 만든다(prefetch).
- 한 칸 생성: 읽기 트랜잭션 → 트랜잭션 밖 생성 → 짧은 쓰기. 쓰기는 advisory lock을 잡고 상태·세트·순번을 다시 확인한 뒤 질문 행과 음성 합성 작업을 등록한다. 같은 프로세스 안에서는 지원별 잠금으로 미리 계산과 즉시 생성이 겹치지 않는다.
- upfront 모드(QUESTION_MODE=upfront)는 10개를 모두 만들고 음성을 미리 합성한다.

3. 회사 자료 RAG — questions/rag_graph.py (함수 루프)

①의 motivation·job_skill 노드 안에서 T2 요청마다 도는 Corrective RAG. 이전에는 LangGraph 하위 그래프였지만, 루프 상태에 문서 원문과 AI 클라이언트가 들어가 체크포인트로 저장하면 안 되고, 복구 단위는 이를 부르는 노드라서 일반 함수로 바꿨다.

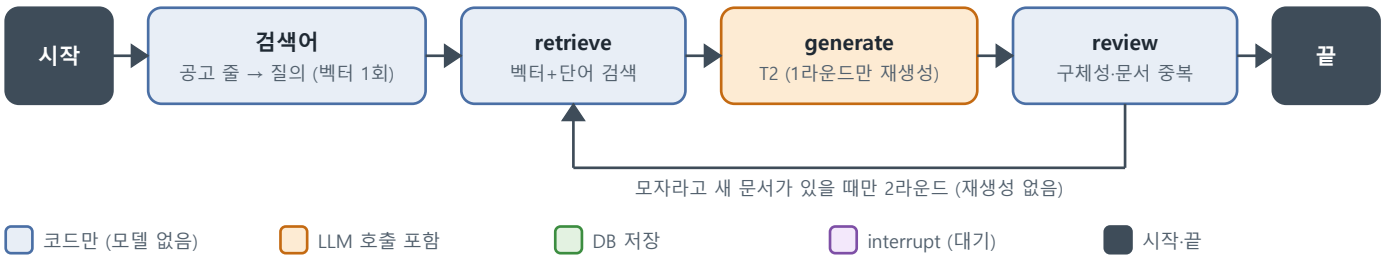


그림 3. 회사 자료 검색·생성 루프. 1라운드가 시간 초과면 2라운드를 하지 않는다. 새 문서가 없으면 1라운드에서 보류한 질문을 다시 검토만 한다(모델 0회).

항목	개선 전	개선 후
한 영역 T2 최악 호출	2라운드 × (1 + 재생성 1 + 형식 2) = 8	1라운드 (1 + 재생성 1 + 형식 1) + 2라운드 (1 + 형식 1) = 5 (오류 없으면 1)
한 영역 임베딩	최대 10회	최대 3회 (질의 벡터 1회를 검색·예시 검색이 함께 사용)
기본 10문항 upfront 최악 LLM	20	14

4. ② 면접 진행 그래프 — interview/graph.py

지원 1건당 그래프 1개(thread_id = 지원 id). 질문 제시 → 답변 대기 → 답변 확정 → 꼬리질문 판단을 질문마다 반복하고, 끝나면 코딩테스트 또는 제출로 넘어간다. 음성 모드와 텍스트 모드가 같은 그래프를 쓴다.

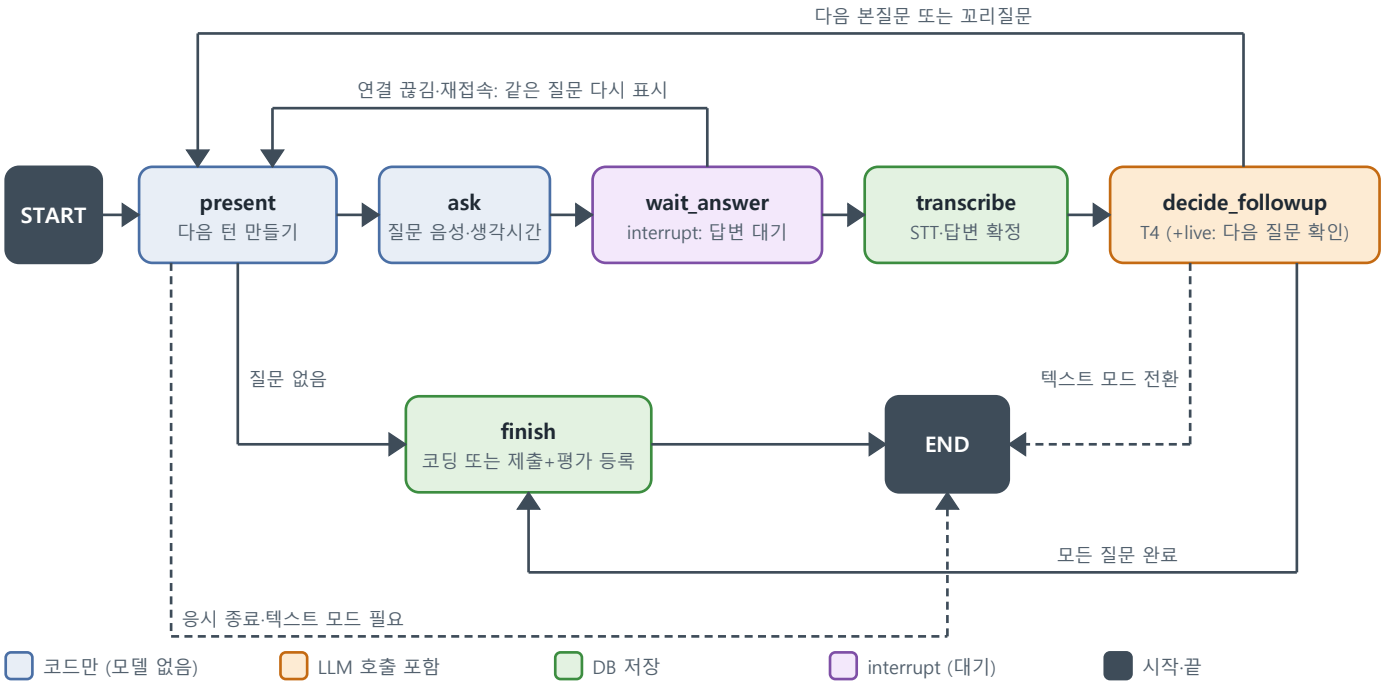


그림 4. 면접 진행 그래프. 점선은 응시 종료·텍스트 모드 전환으로 그래프가 제어를 넘기는 경로.

노드	하는 일
present	다음 턴을 만든다. 꼬리질문이 있으면 그것이 먼저. 남은 질문이 없으면 finish. live 모드면 본질문을 낸 직후 다음 본질문을 미리 만들기 시작
ask	음성 모드: 질문 이벤트 + 미리 합성한 음성 재생 + 생각 시간. 질문 음성이 끝나기 전에는 마이크 입력을 받지 않음
wait_answer	interrupt()로 멈춘 채 대기. 답변 완료 버튼·시간 만료(본질문 30~120초·기본 60초, 꼬리질문 45초)·이탈 신호로 재개. 침묵으로 종료하지 않음
transcribe	답변을 한 번만 확정·저장하고 문항별 사전 평가(pre_evaluate) 등록. 이미 확정된 답변이면 그 id를 그대로 씀
decide_followup	꼬리질문 판단. 이미 판단 기록이 있으면 모델을 부르지 않고 그 결과를 씀. live 모드면 다음 본질문 확인
finish	코딩테스트로 전환하거나, 제출과 평가 작업 등록을 한 트랜잭션으로

중복 방지 (같은 단계가 다시 실행될 때)

- 답변: 질문당 답변 1개(UNIQUE). 확정이 두 번 와도 답변 1개, pre_evaluate 1건.
- 꼬리질문: 본질문당 판단 기록 1개(PK), 꼬리질문 1개(UNIQUE). 쓰기 트랜잭션에서 본질문 행을 잠그고 다시 확인하므로 동시 실행도 IntegrityError 없이 같은 결과.
- 꼬리질문 규칙: 본질문당 1개, 전체 상한(공고 설정 0~4), 무응답·무내용이면 생략. T4는 답변 원문 인용 하나를 근거로 묻는다. 판단 제한 30초, 넘으면 생략.
- 실시간 LLM 호출은 꼬리질문 판단(T4)뿐이고, live 질문 생성은 답변 시간 동안 미리 끝낸다.

5. ③ 평가 그래프 — evaluation/graph.py

제출하면 등록되는 **evaluate_application** 작업이 실행한다. thread_id = eval:<지원 id>:<job id>. 단위(본질문·꼬리질문 답변)는 GPU 6GB를 고려해 하나씩 순차로 평가한다.

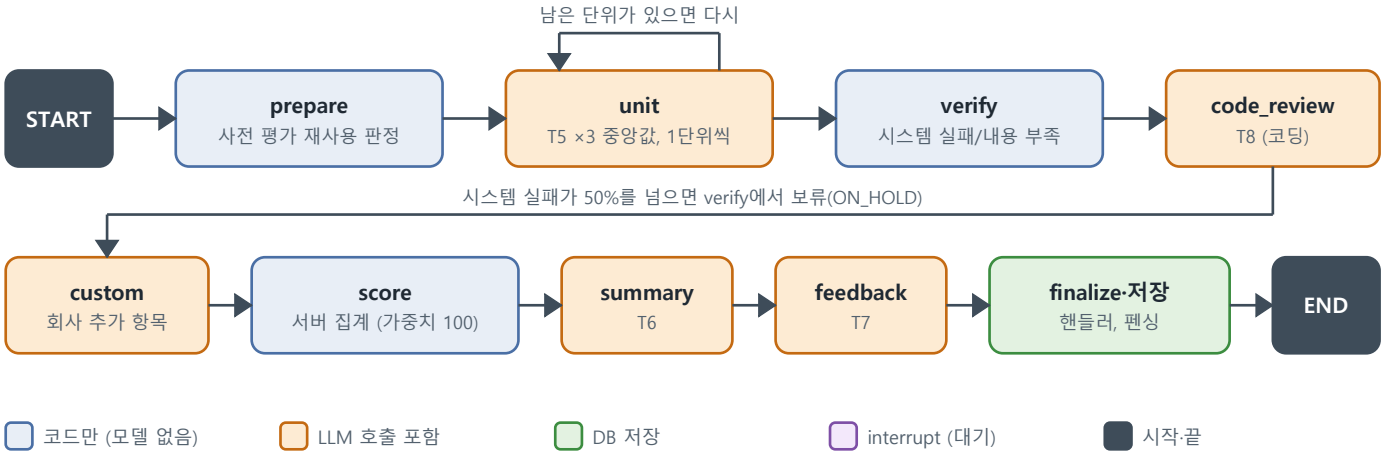


그림 5. 평가 그래프. 그래프가 끝나면 핸들러가 finalize(점수를 다시 계산해 같은지 확인)하고 펜싱 트랜잭션으로 저장한 뒤 체크포인트를 지운다.

노드	하는 일
prepare	사전 평가 결과를 읽고 재사용 규칙 검사: SHAPE(형식), RUBRIC_VERSION, STALE, QUOTE_MISMATCH(인용이 답변과 다름), UNLINKED(3점 이상인데 인용 없음). 모델·프롬프트 버전은 manifest로 확인. 걸린 단위만 재평가 목록에
unit	단위 하나를 T5로 3회 평가해 항목별 중앙값. DB에 먼저 저장하고 체크포인트를 남겨, 사이에 죽어도 다시 부르지 않음
verify	시스템 실패(근거 미연결·인용 불일치·근거 없음)와 내용 부족(무내용·짧음·무응답)을 나눠 센다. 시스템 실패가 50%를 넘으면 보류
code_review · custom	T8 코딩 판단, 회사 추가 평가 항목
score	서버 코드가 확정된 규칙으로 집계(가중치 합 100이 아니면 거부). 점수와 판단 자체는 모델이 정하고 서버는 형식·범위·근거만 검증

노드	하는 일
summary · feedback	T6 담당자 요약, T7 지원자 공개 피드백

- 작업 재시도는 같은 스레드를 이어 실행한다. 측정에서 T7이 형식 오류로 실패했을 때 재시도는 T7만 다시 불렀다.
- manifest가 시도 사이에 바뀌면 처음부터, 실행 도중 바뀌면 시도를 쓰지 않고 지연한다. 서로 다른 버전이 한 리포트에 섞이지 않는다.
- 리포트 detail.evidence에 판정 수, 시스템 실패·내용 부족·무응답 수, 재사용·새 평가·재평가 수가 남는다.

6. 개선 전후 측정 (2026-11-10, 같은 조건 1회)

실제 모델 qwen3.5:4b, eval_samples=3, 합성 데이터 DB 복제본(최신 마이그레이션), 지원 3건 질문 생성 2회 반복, 평가 2건. 도구: tools/bench_ai.sh. 개선 전은 리팩터링 직전 코드 사본.

항목	개선 전	개선 후
질문 생성 소요 p50 / p95	37.3초 / 39.1초	34.3초 / 36.4초
질문 생성 LLM 요청 / 호출(재시도 포함)	22 / 35	18 / 32
질문 생성 임베딩	58	28
트랜잭션 안에서 실행한 모델 호출	80회, 214초	0회
질문 생성 트랜잭션 유지 최대	27.3초 (제한 30초)	35ms
30문항 출처 AI / 코드 대체 / 커스텀	17 / 10 / 3	12 / 15 / 3
사전 평가 소요 p50 / p95, LLM 호출	26.6초 / 29.6초, 63	26.0초 / 29.2초, 63
최종 평가 소요 p50 / p95	13.1초 / 13.4초	24.6초 / 34.9초 (T7 형식 실패 1회 → T7만 재실행)
평가 트랜잭션 최대	92.8ms	51.5ms

- 코드 대체 질문이 늘었다(기업·직무 칸 6개). 2라운드를 새 문서가 있을 때만 돌리고 명확성 검사를 더한 결과로, 호출 수와 질문 다양성의 교환이다.
- 근거 상태 분포(OK 61→52)는 T5 코드·입력이 같아 모델 출력 차이다. 1회 측정이라 반복 측정이 필요하다.

7. 설계 결정 근거

결정	이유	문서
질문 생성·면접 진행·평가를 LangGraph로	단계별 체크포인트로 크래시 후 이어서 실행, 면접 대기를 interrupt로 표현	ADR-0016, 0027
엣지는 코드가 결정	AI가 면접 흐름·합격 여부를 좌우하지 않도록, 검증 가능한 규칙만 분기	ADR-0016
RAG는 함수 루프	문서 원문을 체크포인트에 두지 않고, 호출 수를 줄이기 위해(2라운드는 새 문서가 있을 때만)	ADR-0027
모델 호출 중 트랜잭션 없음	유휴 트랜잭션 30초 제한과 커넥션 점유. 측정에서 최대 27.3초까지 갔음	ADR-0027
기본은 live + 미리 계산	질문이 직전 답변을 반영하도록(사용자 결정). 생성 지연은 답변 시간 동안 미리 만들어 가림	ADR-0022
평가 단위 재사용 규칙	검증된 사전 평가는 다시 부르지 않고, 형식·버전·인용이 어긋난 것만 재평가	ADR-0027

기준 파일: backend/app/questions/graph.py, rag_graph.py, compose.py, live.py, dbio.py, interview/graph.py, interview/turns.py, evaluation/graph.py, evaluation/units.py, evaluation/pipeline.py, jobs/handlers.py, ai/tasks/base.py, ai/tasks/questions.py. 2026-11-10 변경분은 미커밋, api·worker 재빌드 전이다.